Lesson 2 Teacher's Key

Scratch

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

One hat block plus one Motion block:

```
when [right arrow] key pressed
  change x by 10
```

Level 2 - Complete Worked Solution

Four independent scripts, one per arrow key, each a hat block plus one Motion block:

```
when [up arrow] key pressed
  change y by 10

when [left arrow] key pressed
  change x by -10
```

```
when [down arrow] key pressed
  change y by -10

when [right arrow] key pressed
  change x by 10
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

This is the one goal that genuinely needs a loop: Scratch has "when [key] key pressed" as a hat block, but no matching "when [key] key released" block, so there's no event to trigger off of the instant a key goes back up. The only way to know "no direction key is currently held" is to keep checking - which means a separate "when green flag clicked" script with a forever loop, using the Sensing category's "key [ ] pressed?" reporter (a boolean you can poll) rather than the "when key pressed" hat block (an event you can't poll).

The same timer-mod trick as before drives the actual blink once the loop confirms a direction is held: Scratch's Sensing timer reporter counts up in seconds on its own, and (timer mod 0.4) compared against half its own period gives a repeating on/off flip at about 2.5 times a second.

```
when green flag clicked
forever
  if <<<key [right arrow v] pressed?> or <key [left arrow v] pressed?>> or
<<key [up arrow v] pressed?> or <key [down arrow v] pressed?>>> then
    if <(timer mod 0.4) < 0.2> then
      show
    else
```

```
    hide
  else
    show
    set color effect to 0
```

Sound: play on movement, silent when idle

Unlike the blink, this goal doesn't need a loop at all. Upload sfx_blip.wav from this lesson's Assets folder, then drop a single "start sound" block right inside each of the 4 "when key pressed" scripts. Because that script only runs once at the instant of the press, the sound is automatically a one-shot per press and automatically silent whenever no key is being pressed - the same "discrete trigger, no cooldown needed" situation as 001 Game Creator's Input Triggers and Unreal's Enhanced Input Action Events elsewhere in this lesson, not the continuous-loop-with-a-timer-gate that Godot, Unity, GameMaker, GDevelop, and TIC-80 need because their movement is polled every frame instead of event-triggered.

```
when [right arrow] key pressed
change x by 10
start sound [sfx_blip]
```

Recolor: different costume per direction, reset when idle

Scratch sprites can hold multiple Costumes, so this goal swaps the whole picture rather than tinting one: upload player_circle_red.png, _blue.png, _green.png, and _yellow.png as extra Costumes on your sprite (Costumes tab > Upload Costume), plus player_circle_orange.png as a dedicated idle costume - even if Setup had you start with a different color, treat orange as the one true idle appearance from here on, so idle never gets confused with a direction that happens to reuse your original color.

"switch costume to [costume]" (Looks) takes a different costume per direction; switch it the moment a direction key is pressed, right inside that key's own script.

Switching back to orange when idle has the same problem as the blink: there's no "key released" event to trigger it. Rather than inventing a second loop, reuse the exact loop already built for the blink above - it's already checking "is any direction key currently held" every frame, so the idle branch is the natural place to also switch back to the idle costume.

```
-- inside each "when [direction] key pressed" script:
switch costume to [player_circle_red]   -- (blue for left, green for up,
yellow for down)

-- inside the green-flag loop's else branch (idle), alongside show:
switch costume to [player_circle_orange]
```

Bounce off the edges

"if on edge, bounce" (Motion) is Scratch's purpose-built edge-collision block, and it's a better fit here than manually checking x position/y position against the stage's bounds - one block instead of four separate if-checks, and it's the block most Scratch tutorials reach for first.

Because movement in this version happens one discrete step at a time (each key press moves 10 steps, not a smooth continuous slide), the bounce check only needs to run once, right after that step - no forever loop required. Drop it directly after the "change x/y by" block in each of the 4 key-press scripts.

Known quirk, okay for now: "if on edge, bounce" also reverses the sprite's direction (heading), and if a sprite's rotation style is left-right (the default for many costumes), that direction change

shows up as the sprite flipping horizontally when it bounces off a side edge. That's expected Scratch behavior, not a bug - precise, non-flipping boundary control is a later-lesson upgrade (the manual position-checking technique), not something to chase down this week.

```
when [right arrow] key pressed
change x by 10
if on edge, bounce
```

Structure: group everything triggered by the same action in one place

The most important structural choice in this version: stay with 5 separate scripts - one "when green flag clicked" and one "when [key] key pressed" per arrow key - instead of collapsing everything into a single forever loop. Scratch is fundamentally event-driven, and "when key pressed" hat blocks are the idiomatic way to react to input; a forever loop that polls "key pressed?" every frame is the right tool only for the one thing that genuinely can't be event-driven here (detecting "no key is held," for the blink/idle-reset).

Each of the 4 key-press scripts becomes its own self-contained "if this key, do this" - movement, sound, recolor, and bounce for that direction all live together in the one script that direction's key press triggers, so there's exactly one place to look for everything the right arrow does. That's what "the 4 key-press scripts effectively become the if-then-else structure" means: the branching is the four separate hat blocks themselves, not an if/elseif/else chain inside one shared script.

```
when [right arrow] key pressed
  change x by 10
  if on edge, bounce
  set color effect to 50
  start sound [sfx_blip]

when [up arrow] key pressed
  change y by 10
  if on edge, bounce
  set color effect to 150
  start sound [sfx_blip]
```

```
when [left arrow] key pressed
  change x by -10
  if on edge, bounce
  set color effect to 100
  start sound [sfx_blip]

when [down arrow] key pressed
  change y by -10
  if on edge, bounce
  set color effect to 25
  start sound [sfx_blip]
```

Complete Level 3 solution (all 5 goals combined)

The full 5-script solution: one "when green flag clicked" script handling blink and idle color-reset (the only two things that need continuous checking), plus 4 independent "when key pressed" scripts, each handling that direction's movement, bounce, color, and sound in one place.

```
when green flag clicked
forever
  if <<<key [right arrow v] pressed?> or <key [left arrow v] pressed?>> or
<<key [up arrow v] pressed?> or <key [down arrow v] pressed?>>> then
    if <(timer mod 0.4) < 0.2> then
      show
    else
      hide
  else
    show
    switch costume to [player_circle_orange]

when [right arrow] key pressed
  change x by 10
  if on edge, bounce
```

```
  switch costume to [player_circle_red]
  start sound [sfx_blip]

when [left arrow] key pressed
  change x by -10
  if on edge, bounce
  switch costume to [player_circle_blue]
  start sound [sfx_blip]

when [up arrow] key pressed
  change y by 10
  if on edge, bounce
  switch costume to [player_circle_green]
  start sound [sfx_blip]

when [down arrow] key pressed
  change y by -10
  if on edge, bounce
  switch costume to [player_circle_yellow]
  start sound [sfx_blip]
```